

Introduction to {fmt}/std::format

C++ User Group

2026-02-04

C++ User-Treffen in Aachen am 14.1.2016



Foto 08: Daniel Evers mit "String-Formatierungen mit TMP"

```
53      const char* x = "x";
54
55      auto f = build_format_string("Hello ", str(), " #", num(),
56      std::cout << f((const char*)"world", 5);
57      std::cout << f("world", 7L);
58      std::cout << f(x, (unsigned int)88);
59      std::cout << f(x, (int8_t)-127);
60      std::cout << f("world", -1);
61      std::cout << f("world", (uint8_t)9);
62  //    std::cout << f();
63  //    std::cout << f(1, 2);
64  //    std::cout << f(5);
65
66      auto f1 = build_format_string("Pointer value: ", ptr());
67      std::cout << f1(&f1) << "\n";
68      std::cout << f1(x) << "\n";
69      std::cout << f1(nullptr) << "\n";
70
71      auto f2 = build_format_string();
```

http://www.wilkening-online.de/programmieren/c++-user-treffen-aachen/2016_01_14/bericht-2016-01-14.html

To print or to stream?

	Pros	Cons
printf	Easier to read Easier to search for Relatively fast	Not extensible Horrible security history Error prone, complex Pure C
ostream	Extensible C++	Hard to read Hard to search for Usually "not fast“ I/O manipulators

String formatting in C++: {fmt}

- Started in early 2014 by Victor Zverovich
- Hundreds of contributors
- Highly portable, requires a minimal subset of C++11
- Syntax inspired by Python's formatting
- (Partly) standardized in C++20
  - `std::print` added in C++23

{fmt} examples

```
fmt::print("The answer is {}\n", 42);

fmt::print("The answer is {}\n", "somewhere else");

std::string result = fmt::format("3 + 4 = {:d}", 3 + 4);

std::string hexres = fmt::format("3 + 4 = {:x}", 3 + 4);

// C++23
std::println("Hello World!");
```

{fmt} format string syntax

```
"First, thou shalt count to {0}" // References the first argument
"Bring me a {}"                  // Implicitly references the first argument
"From {} to {}"                  // Same as "From {0} to {1}"

fmt::format("{:>10}", "Hello")

replacement_field ::= "{" [arg_id] [":" spec] "}"

// spec can be customized for custom types;
// most common options:
format_spec ::= [[fill]align][sign]["#"]["0"][width]["." precision]["L"][type]
fill        ::= <a character other than '{' or '}'>
align       ::= "<" | ">" | "^"
sign        ::= "+" | "-" | " "
width       ::= integer | "{" [arg_id] "}"
precision   ::= integer | "{" [arg_id] "}"
type        ::= "a" | "A" | "b" | "B" | "c" | "d" | "e" | "E" | "f" | "F" |
                "g" | "G" | "o" | "p" | "s" | "x" | "X" | "?"
```

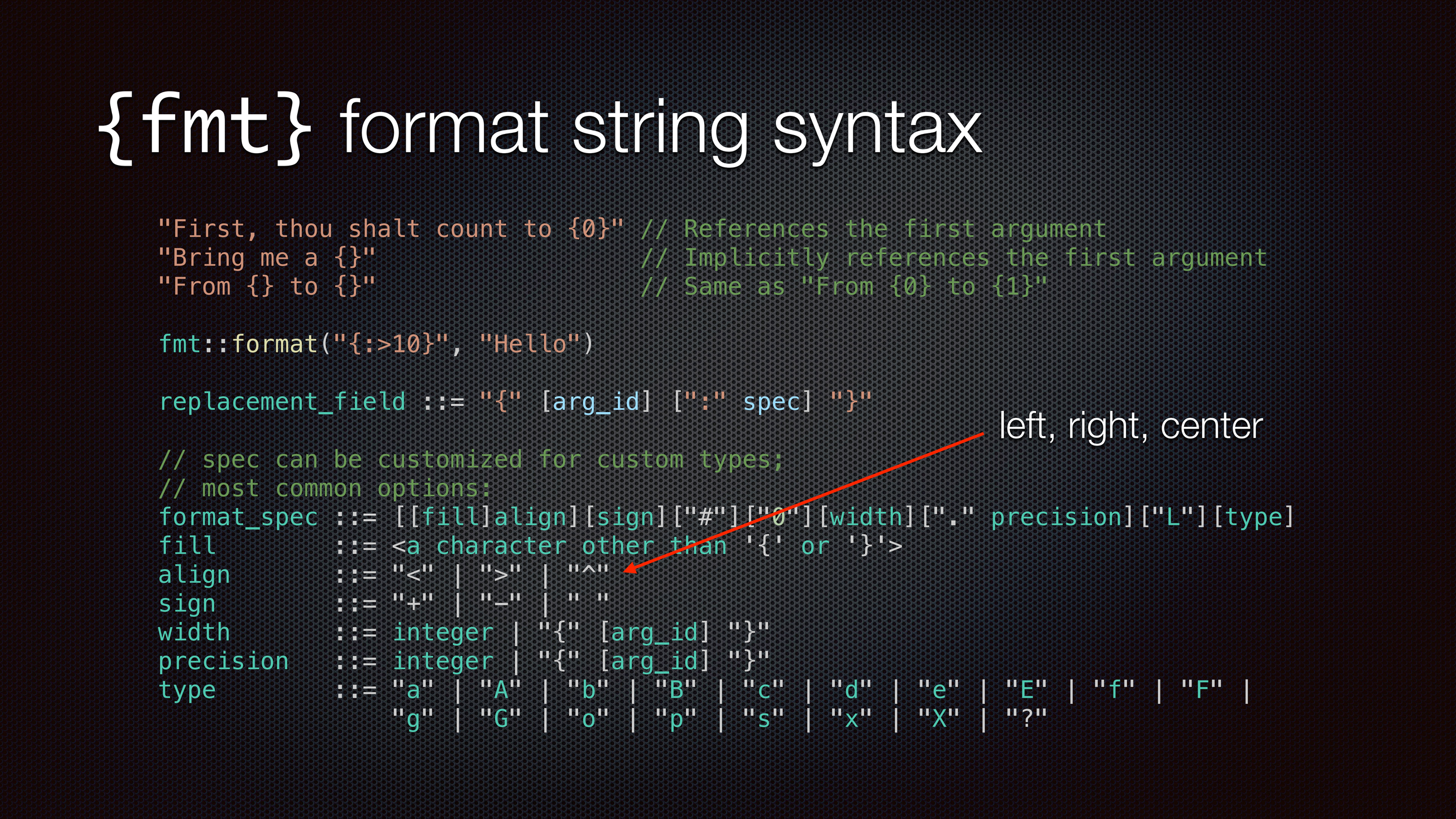
{fmt} format string syntax

```
"First, thou shalt count to {0}" // References the first argument
"Bring me a {}"                  // Implicitly references the first argument
"From {} to {}"                  // Same as "From {0} to {1}"

fmt::format("{:>10}", "Hello")

replacement_field ::= "{" [arg_id] [":" spec] "}"

// spec can be customized for custom types;
// most common options:
format_spec ::= [[fill]align][sign]["#"]["0"][width]["." precision]["L"][type]
fill        ::= <a character other than '{' or '}'>
align       ::= "<" | ">" | "^"
sign        ::= "+" | "-" | " "
width       ::= integer | "{" [arg_id] "}"
precision   ::= integer | "{" [arg_id] "}"
type        ::= "a" | "A" | "b" | "B" | "c" | "d" | "e" | "E" | "f" | "F" |
                "g" | "G" | "o" | "p" | "s" | "x" | "X" | "?"
```



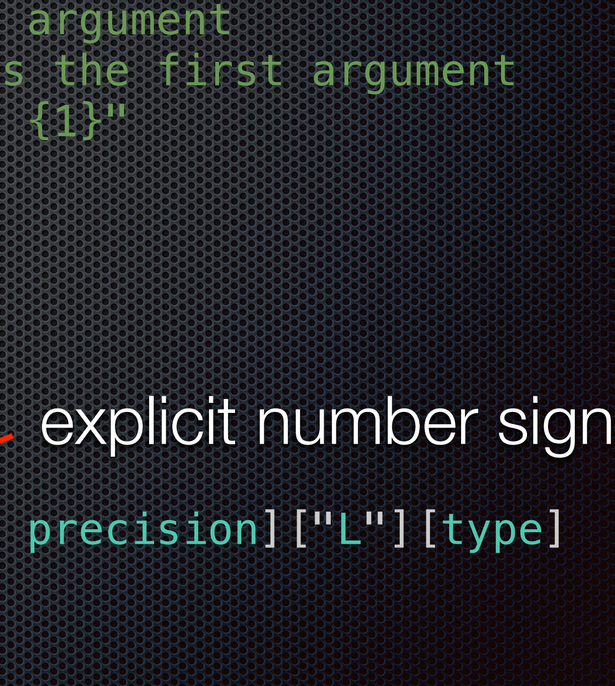
{fmt} format string syntax

```
"First, thou shalt count to {0}" // References the first argument
"Bring me a {}"                  // Implicitly references the first argument
"From {} to {}"                  // Same as "From {0} to {1}"

fmt::format("{:>10}", "Hello")

replacement_field ::= "{" [arg_id] [":" spec] "}"

// spec can be customized for custom types;
// most common options:
format_spec ::= [[fill]align][sign]["#"]["0"][width]["." precision]["L"][type]
fill        ::= <a character other than '{' or '}'>
align       ::= "<" | ">" | "^"
sign        ::= "+" | "-" | " "
width       ::= integer | "{" [arg_id] "}"
precision   ::= integer | "{" [arg_id] "}"
type        ::= "a" | "A" | "b" | "B" | "c" | "d" | "e" | "E" | "f" | "F" |
                "g" | "G" | "o" | "p" | "s" | "x" | "X" | "?"
```



{fmt} format string syntax

```
"First, thou shalt count to {0}" // References the first argument
"Bring me a {}"                  // Implicitly references the first argument
"From {} to {}"                  // Same as "From {0} to {1}"

fmt::format("{:>10}", "Hello")

replacement_field ::= "{" [arg_id] [":" spec] "}"

// spec can be customized for custom types;
// most common options:
format_spec ::= [[fill]align][sign]["#"]["0"][width]["." precision]["L"][type]
fill        ::= <a character other than '{' or '}'>
align       ::= "<" | ">" | "^"
sign        ::= "+" | "-" | " "
width       ::= integer | "{" [arg_id] "}"
precision   ::= integer | "{" [arg_id] "}"
type        ::= "a" | "A" | "b" | "B" | "c" | "d" | "e" | "E" | "f" | "F" |
                "g" | "G" | "o" | "p" | "s" | "x" | "X" | "?"
```

Alternate form (`0b`/`0x`, decimal point, …)

{fmt} format string syntax

```
"First, thou shalt count to {0}" // References the first argument
"Bring me a {}"                  // Implicitly references the first argument
"From {} to {}"                  // Same as "From {0} to {1}"

fmt::format("{:>10}", "Hello")

replacement_field ::= "{" [arg_id] [":" spec] "}"

// spec can be customized for custom types;
// most common options:
format_spec ::= [[fill]align][sign]["#"]["0"][width]["." precision]["L"][type]
fill        ::= <a character other than '{' or '}'>
align       ::= "<" | ">" | "^"
sign        ::= "+" | "-" | " "
width       ::= integer | "{" [arg_id] "}"
precision   ::= integer | "{" [arg_id] "}"
type        ::= "a" | "A" | "b" | "B" | "c" | "d" | "e" | "E" | "f" | "F" |
                "g" | "G" | "o" | "p" | "s" | "x" | "X" | "?"
```



Use locale for number separator

{fmt} format string syntax

```
type        ::= "a" | "A" | "b" | "B" | "c" | "d" | "e" | "E" | "f" | "F" |
                "g" | "G" | "o" | "p" | "s" | "x" | "X" | "?"
```

- b/B, c, d, o, x/X: integer presentation (binary, octal, character, …)
- a/A, e/E, f/F, g/G: floating-point representation (hex, exponent, fixed, …)
- c/s: character/string
- ?: debug format for characters & strings (quoted & escaped)
- p: pointers

{fmt} examples

```
fmt::format("{0}, {1}, {2}", 'a', 'b', 'c');
// Result: "a, b, c"
fmt::format("{}, {}, {}", 'a', 'b', 'c');
// Result: "a, b, c"
fmt::format("{2}, {1}, {0}", 'a', 'b', 'c');
// Result: "c, b, a"
fmt::format("{0}{1}{0}", "abra", "cad");  // arguments' indices can be repeated
// Result: "abracadabra"
```

{fmt} examples

```
fmt::format("{:<30}", "left aligned");
// Result: "left aligned                  "
fmt::format("{:>30}", "right aligned");
// Result: "                 right aligned"
fmt::format("{:^30}", "centered");
// Result: "           centered             "
fmt::format("{:*^30}", "centered");  // use '*' as a fill char
// Result: "**********centered**********"

fmt::format("{:<{}}", "left aligned", 30);
// Result: "left aligned
```

{fmt} features: Safety

```
// compile-time format string checks
// - compiler error
fmt::println("{:d}", "I am not a number");
// - runtime exception
fmt::println(fmt::runtime("{:d}"), "I am not a number");

// automatic memory management
std::string s = fmt::format("x is {0:d} (hex {0:x})", x);
```

{fmt} features: Standard types

```
// Supports ranges, tuples, date & time, variant, atomic, ...
auto v = std::vector{1, 2, 3};
fmt::print("{}", v);                          // [1, 2, 3]
fmt::print("{:02}", fmt::join(v, ", ")); // 01, 02, 03

fmt::print("Today is {:%Y-%m-%d}.\n", std::chrono::system_clock::now());
```

{fmt} features: Standard types

```
// Supports ranges, tuples, date & time, variant, atomic, ...
auto v = std::vector{1, 2, 3};
fmt::print("{}", v);                          // [1, 2, 3]
fmt::print("{:02}", fmt::join(v, ", ")); // 01, 02, 03

fmt::print("Today is {:%Y-%m-%d}.\n", std::chrono::system_clock::now());
```

Dedicated format string syntax for chrono types, durations, ranges

{fmt} format string syntax: ranges

```
range_format_spec ::= ["n"][range_type][range_underlying_spec]
```

- `n`: omit opening & closing brackets
- `range_type`: empty, `s` or `?s` (for strings only)
- `range_underlying_spec`: parsed based on the range's element type

{fmt} features: Extensibility

```
// use format_as() if there's a similar, existing type
struct my_string_type { /* ... */ };
auto format_as(const my_string_type& t) { return std::string_view(t); }

struct Meters { int value = 0; };
auto format_as(const Meters& m) { return m.value; }
```

{fmt} features: Extensibility

```
// specialize formatter<> for custom formatting

enum class color {red, green, blue};


template <> struct fmt::formatter<color>: formatter<string_view> {
  // parse is inherited from formatter<string_view>.

  auto format(color c, format_context& ctx) const {
    string_view name = "unknown";
    switch (c) {
        case color::red:   name = "red"  ; break;
        case color::green: name = "green"; break;
        case color::blue:  name = "blue" ; break;
    }
    return formatter<string_view>::format(name, ctx);
  }
};
```

{fmt} features: Extensibility

```
// specialize formatter<> for custom formatting

enum class color {red, green, blue};

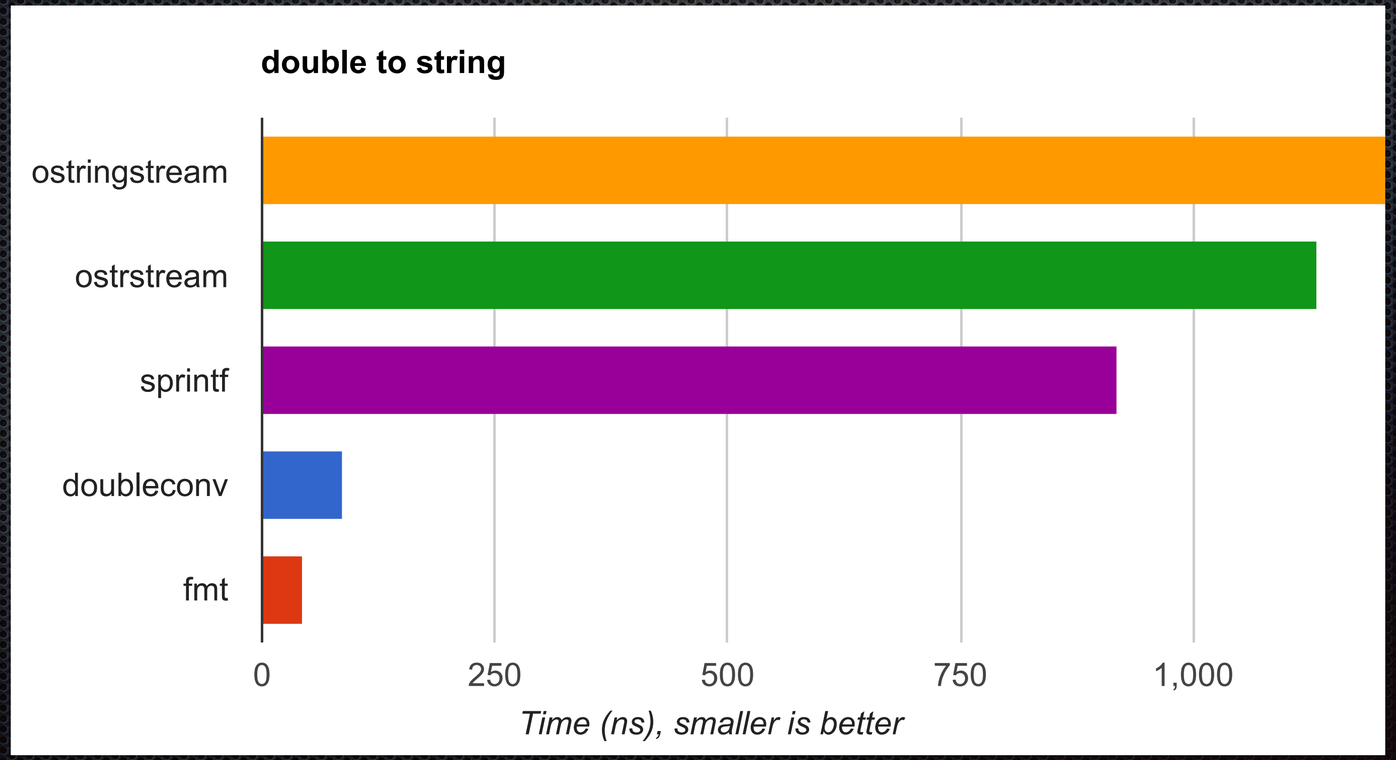

template <> struct fmt::formatter<color>: formatter<string_view> {
  // parse is inherited from formatter<string_view>.

  auto format(color c, format_context& ctx) const {
    string_view name = "unknown";
    switch (c) {
        case color::red:   name = "red"  ; break;
        case color::green: name = "green"; break;
        case color::blue:  name = "blue" ; break;
    }
    return formatter<string_view>::format(name, ctx);
  }
};
```

Implement parse() for custom formatting options

{fmt} features: Performance

- Can be MUCH faster than `iostreams` and `sprintf`, esp. for numeric formatting



{fmt} features: Build speed & size

Optimized build (-O3)

Method	Compile Time, s	Executable size, KiB	Stripped size, KiB
printf	1.6	54	50
IOStreams	28.4	98	84
{fmt} 1122268	5.0	54	50
tinyformat	32.6	164	136
Boost Format	55.0	530	317

{fmt} features: Build speed & size

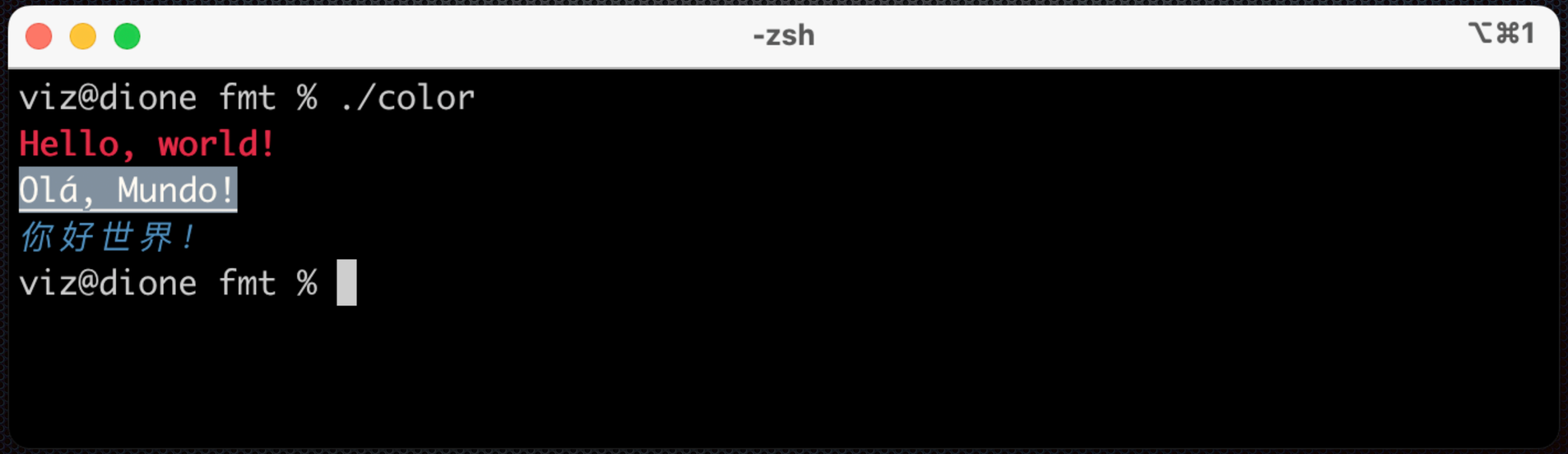
Non-optimized build

Method	Compile Time, s	Executable size, KiB	Stripped size, KiB
printf	1.4	54	50
IOStreams	27.0	88	68
{fmt} 1122268	4.7	87	84
tinyformat	28.1	185	145
Boost Format	38.9	678	381

`{fmt}` features: Others

- Portable Unicode support (on major operating systems with UTF-8)
- Optional locale support
- Terminal color and text style support
- Contains a safe `printf` implementation

```cpp
fmt::print(fg(fmt::color::crimson) | fmt::emphasis::bold,
           "Hello, {}!\n", "world");
fmt::print(fg(fmt::color::floral_white) | bg(fmt::color::slate_gray) |
           fmt::emphasis::underline, "Olá, {}!\n", "Mundo");
fmt::print(fg(fmt::color::steel_blue) | fmt::emphasis::italic,
           "你好{}! \n", "世界");
```

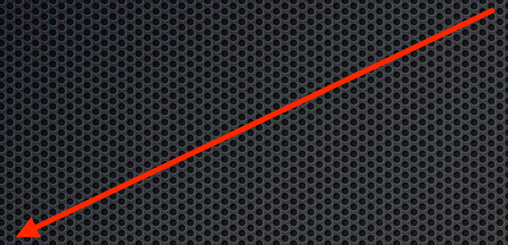


Try it!

- https://fmt.dev/
- https://en.cppreference.com/w/cpp/utility/format.html

Try it!

Try `fmt::to_string(x)`!



- https://fmt.dev/
- https://en.cppreference.com/w/cpp/utility/format.html